NETWORK SECURITY LAB MANUAL

PART - A

Program 1: Caesar Cipher

Aim

To implement Caesar Cipher substitution technique using Python.

Algorithm

1. Read the plaintext.
2. Read the shift value.
3. Encrypt by shifting each alphabet.
4. Decrypt by reversing the shift.
5. Display encrypted and decrypted text.

Python Program

```
def encrypt(text, shift):
    result = ""
    for char in text:
        if char.isalpha():
            if char.isupper():
                result += chr((ord(char) - 65 + shift) % 26 + 65)
            else:
                result += chr((ord(char) - 97 + shift) % 26 + 97)
        else:
            result += char
    return result

def decrypt(text, shift):
    return encrypt(text, -shift)

text = input("Enter Plain Text: ")
shift = int(input("Enter Shift Value: "))
cipher = encrypt(text, shift)
print("Encrypted Text:", cipher)
plain = decrypt(cipher, shift)
print("Decrypted Text:", plain)
```

Sample Output

```
Enter Plain Text: HELLO
Enter Shift Value: 3
Encrypted Text: KHOOR
Decrypted Text: HELLO
```

Result

Thus Caesar Cipher was successfully implemented.

Program 2: Mono Alphabetic Cipher

Aim

To implement Mono Alphabetic Cipher using a key.

Python Program

```
alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
key = "QWERTYUIOPASDFGHJKLZXCVBNM"
plain = input("Enter Plain Text: ").upper()
cipher = ""

for ch in plain:
    if ch in alphabet:
        cipher += key[alphabet.index(ch)]
    else:
        cipher += ch
print("Encrypted:", cipher)

decrypt = ""
for ch in cipher:
    if ch in key:
        decrypt += alphabet[key.index(ch)]
    else:
        decrypt += ch
print("Decrypted:", decrypt)
```

Sample Output

```
Enter Plain Text: HELLO
Encrypted: ITSSG
Decrypted: HELLO
```

Program 3: Vigenere (Poly Alphabetic) Cipher

Python Program

```
def generateKey(text, key):
    key = list(key)
    if len(text) == len(key):
        return("".join(key))
    else:
        for i in range(len(text) - len(key)):
            key.append(key[i % len(key)])
        return("".join(key))

def encrypt(text, key):
    cipher = []
    for i in range(len(text)):
        x = (ord(text[i]) + ord(key[i])) % 26
        x += 65
        cipher.append(chr(x))
    return("".join(cipher))

def decrypt(cipher, key):
    plain = []
    for i in range(len(cipher)):
        x = (ord(cipher[i]) - ord(key[i]) + 26) % 26
        x += 65
        plain.append(chr(x))
    return("".join(plain))

text = input("Enter Text: ").upper()
key = input("Enter Key: ").upper()
key = generateKey(text, key)
cipher = encrypt(text, key)
print("Encrypted:", cipher)
plain = decrypt(cipher, key)
print("Decrypted:", plain)
```

Sample Output

```
Enter Text: hello
Enter Key: 3
Encrypted: TQXXA
Decrypted: HELLO
```

Program 4: Playfair Cipher

Python Program

```
def encrypt(text):
    text = text.upper().replace(" ", "")
    if len(text) % 2 != 0:
        text += "X"
    cipher = ""
    for i in range(0, len(text), 2):
        cipher += chr((ord(text[i]) - 65 + 3) % 26 + 65)
        cipher += chr((ord(text[i+1]) - 65 + 3) % 26 + 65)
    return cipher

plain = input("Enter Message: ")
cipher = encrypt(plain)
print("Encrypted:", cipher)
```

Sample Output

```
Enter Message: Hello Mysore
Encrypted: KHOORPBVRUHA
```

Program 5: Hill Cipher

Python Program

```
import numpy as np

text = input("Enter Text (4 letters): ").upper()
if len(text) % 2 != 0:
    text += "X"
cipher = ""
for i in range(0, len(text), 2):
    pair = np.array([[ord(text[i]) - 65], [ord(text[i+1]) - 65]])
    result = np.dot(key, pair) % 26
    cipher += chr(result[0][0] + 65)
    cipher += chr(result[1][0] + 65)
print("Encrypted:", cipher)
```

Sample Output

```
Enter Text (4 letters): PYTHON
Encrypted: NUAVDP
```

Program 6: Single & Double Transposition Cipher

Python Program

```
plain = input("Enter Text: ")
single = plain[::-1]
print("Single Transposition:", single)
double = single[::-1]
print("Double Transposition:", double)
```

Sample Output

```
Enter Text: Hello
Single Transposition: olleh
Double Transposition: Hello
```

PART - B

Program 7: Simplified DES

```
Installation required: pip install pycryptodome
```

Python Program

```
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad

key = b'12345678'
cipher = DES.new(key, DES.MODE_ECB)
text = input("Enter Text: ")
encrypted = cipher.encrypt(pad(text.encode(), 8))
print("Encrypted:", encrypted.hex())
decrypted = unpad(cipher.decrypt(encrypted), 8)
print("Decrypted:", decrypted.decode())
```

Sample Output

```
Enter Text: HELLO INDIA
Encrypted: 45a6a83718e37b202aedd7b6dbla3cla
Decrypted: HELLO INDIA
```

Program 8: AES Encryption

Python Program

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad

key = b'0123456789abcdef'
cipher = AES.new(key, AES.MODE_ECB)
text = input("Enter Message: ")
encrypted = cipher.encrypt(pad(text.encode(), 16))
print("Encrypted:", encrypted.hex())
decrypted = unpad(cipher.decrypt(encrypted), 16)
print("Decrypted:", decrypted.decode())
```

Sample Output

```
Enter Message: HELLO MYSORE
Encrypted: 44a17cbb9d1cebb0b4b540b45cdea3a4
Decrypted: HELLO MYSORE
```

Program 9: RSA Algorithm

Python Program

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP

key = RSA.generate(2048)
public = key.publickey()
cipher = PKCS1_OAEP.new(public)
text = input("Enter Message: ")
encrypted = cipher.encrypt(text.encode())
print("Encrypted:", encrypted.hex())
decrypt = PKCS1_OAEP.new(key)
plain = decrypt.decrypt(encrypted)
print("Decrypted:", plain.decode())
```

Sample Output

```
Enter Message: INDIA IS OUR COUNTRY
Encrypted:
7d7989f82bc3aee590691889282ae2a39361f248c05b4fb8033a28248e31bdc34315071a850e9b55ebeafb
3dd33c20fa281e2b0adb26f978e2f4194cceefcebfe4190474d508744ec7cb4a965f13d7419ac94a706708
2f4f8b0849a34c2cd2befae6218b0b39a4a3cfb5cf56fab5268fd59e00c83556a69ca03878e8f85abb7d27
ee457596ec3c0d06b6c44fe4fe6d072435e719d9d3304170f505d28a177dad47a30cff9b13305519ff99b5
c874353c5ae437c9eb6c994d4224f19c34e8e185191bd75294666elf0fa692739a03b4e165cd4elda29de2
```

```
f81ed5c4f006e575d2a47c70f015c4b65c1609d9592ce169184973e1b7564d6af32d5b91e06f08cffc
Decrypted: INDIA IS OUR COUNTRY
```

Program 10: Digital Signature

Python Program

```
from Crypto.PublicKey import RSA
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256

key = RSA.generate(2048)
msg = input("Enter Message: ")
hash = SHA256.new(msg.encode())
signature = pkcs1_15.new(key).sign(hash)
print("Signature Generated")

try:
    pkcs1_15.new(key.publickey()).verify(hash, signature)
    print("Signature Verified")
except:
    print("Verification Failed")
```

Sample Output

```
Enter Message: HELLO NETWORK SECURITY
Signature Generated
Signature Verified
```

Program 11: Linear Congruential Generator

Python Program

```
a = 5
c = 3
m = 16

x = int(input("Enter Seed: "))
n = int(input("How many numbers: "))

for i in range(n):
    x = (a * x + c) % m
    print(x)
```

Sample Output

```
Enter Seed: 2
How many numbers: 10
13
4
7
6
1
8
11
10
5
12
```

Program 12: Diffie-Hellman Key Exchange

Python Program

```
p = 23
g = 5

a = int(input("Enter Private Key of A: "))
b = int(input("Enter Private Key of B: "))

A = (g ** a) % p
B = (g ** b) % p

print("Public Key A:", A)
print("Public Key B:", B)

keyA = (B ** a) % p
keyB = (A ** b) % p

print("Shared Key at A:", keyA)
print("Shared Key at B:", keyB)
```

Sample Output

```
Enter Private Key of A: 6
Enter Private Key of B: 15
Public Key A: 8
Public Key B: 19
Shared Key at A: 2
Shared Key at B: 2
```